

BHTTP/1: HTTP, in binary

Protocol specification, version 1. Lavya Tanotra 24BCS10124. The keywords MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. HEXDUMP.md walks through a full exchange byte by byte.

1. Conventions

BHTTP/1 transfers files from a server to a client over a single long-lived TCP connection (default port 9000). All integers are unsigned and big-endian. Strings are UTF-8, carry an explicit length and are never NUL-terminated.

2. Connection

The client opens one TCP connection and sends the preface 42 48 54 31 ("BHT1"). From then on, both sides send only frames. If a server reads any other preface, it sends GOAWAY(3) and closes. The client MUST NOT open a second connection, with one exception: if it sent a newer preface (for example "BHT2") and got GOAWAY(3) back, it MAY reconnect once using "BHT1". That is the upgrade path for version 2.

Requests are strictly one at a time: the client MUST NOT send a request before the previous response has ended (END_STREAM), so no more than one stream is ever open. If a server receives a request early anyway, it MAY answer it in order or send GOAWAY(1). The connection stays open after each response, and either side ends it with GOAWAY (section 4). A server MAY close an idle connection with GOAWAY(0), and MAY close without GOAWAY before it has read the preface (slow preface, server full).

3. Frame header (8 bytes, fixed)

Length (24 bits) | Type (8) | Flags (8) | Stream ID (24), followed by Length bytes of payload. That is 64 bits, which shows up as two aligned 32-bit words in a hexdump. Section 4 lists the Type values.

Length: the payload size in bytes, not counting the header. Senders MUST NOT go above 16384, so no single file can monopolise the connection and a receiver never has to buffer more than 8 + 16384 bytes for one frame. A receiver checks Length before it looks at Type: if Length is above 16384, on any type, known or unknown, it sends GOAWAY(2) and closes without reading the payload. If Length is 16384 or less it MUST read the whole frame. Using 24 bits means a later version can raise the cap without changing the header.

Flags: the only flag is 0x01 END_STREAM. Senders MUST set every other bit to 0, and receivers MUST ignore them.

Stream ID: links a response to its request. The client numbers its requests from 1, always increasing (gaps are allowed), and the server answers on the same ID. A stream is *idle* until its request is sent, *open* until the response frame carrying END_STREAM, and *closed* after that. A lower ID that was skipped counts as closed too: the only frame ever sent on a stream that isn't open is the 400 answering it (section 9). ID 0 is reserved for GOAWAY. When the IDs run out (16,777,215) the client sends GOAWAY(0).

Why these widths. Type and Flags take one byte each, which keeps every field byte-aligned; 256 types leave room for version 2, and only one flag is needed today. Stream ID is 24 bits rather than HTTP/2's 31 bits plus a reserved bit. With only one stream open at a time, the ID is there to catch stale or misdirected frames, not to multiplex, and 24 bits keep the header at exactly 8 bytes.

4. Frame types

0x0 HEADERS: a single request (client) or a response status plus headers (server), always in one frame.

0x1 DATA: response body bytes. A response can have zero or more DATA frames, and a zero-length one is legal.

0x2 GOAWAY: Length MUST be 2 and Stream ID MUST be 0; anything else is a protocol error. The payload is an error code: 0 normal, 1 protocol error, 2 frame too large, 3 bad preface, 4 internal error. A receiver treats an unknown code as 1. Once an endpoint has sent or received GOAWAY it MUST NOT send anything else on the connection. A request still open at that point was not completed and is not retried.

Unknown types (Length within the section 3 cap; over it is GOAWAY(2)): a receiver MUST read the Length bytes, throw them away and carry on, whatever the Stream ID or Flags say. It MUST NOT treat this as an error. This is how version 2 can add frame types. A server likewise skips any DATA frame from a client.

5. Request (HEADERS payload, client to server)

Method (1) | Path length (2) | Path | Header count (1) | Header entries

Method: 0x01 GET, 0x02 HEAD; any other value is malformed. The client MUST set END_STREAM. Since a request is always exactly one frame, the server ignores the flag. Requests never have a body. Every request header is optional, and a server MUST NOT reject a request because one is missing.

Path: UTF-8, starts with "/", contains no NUL, at most 16380 bytes, and is used literally: no query string and no percent-decoding ("%2e%2e" is just a file name). A "." segment is malformed. The server resolves a path in this order: (1) drop empty and "." segments (//a/. /b becomes /a/b); (2) map "/" and directories to their index.html; (3) if the path as sent ends in "/" and names a regular file, answer 404. So /index.html/ is 404 while /index.html/. is the file. A missing or non-regular file, and anything that resolves outside the server root (symlinks included), is 404.

A request MAY include the literal header if-none-match. If its value equals the file's etag, or is *, the answer is 304.

6. Response (HEADERS payload, server to client)

Status (2) | Header count (1) | Header entries

Status uses the HTTP numbers and is always final; there are no interim 1xx responses. bserve sends 200, 304, 400, 404 and 500. A client MUST treat a status outside 100-599 as a protocol error. Every response carries content-length, written as one to 18 ASCII digits 0-9 (leading zeros allowed); receivers MUST reject any other value as a protocol error. A 200 or 304 also carries content-type, last-modified, etag and cache-control, and every response carries server and date. Dates are IMF-fixdate (RFC 9110 section 5.6.7).

When there is a body, HEADERS has no flags, DATA frames follow, and the last DATA frame carries END_STREAM; the DATA lengths MUST add up to content-length. When there is no body (HEAD, 304, an empty file, every error), END_STREAM goes on HEADERS and no DATA follows. For HEAD and 304, content-length is the size a GET would return; errors carry content-length 0. Senders MUST put END_STREAM on HEADERS when the body is empty, but receivers also accept a single empty DATA frame carrying END_STREAM.

7. Header entries (the first two ideas from HPACK)

Every entry starts with a 1-byte index. 0x01-0x0A: a name from the static table, then Value length (2) and Value. 0x00: a literal name, given as Name length (1, at least 1), Name, Value length (2), Value. 0x0B-0xFF are reserved and therefore malformed.

1 host	2 user-agent	3 accept	4 content-type	5 content-length
6 server	7 date	8 etag	9 last-modified	10 cache-control

bcurl sends 1-3 and bserve sends 4-10. Names use only ASCII a-z 0-9 -. A name MUST NOT appear twice (a literal that equals a static name counts as that name), order does not matter, and receivers ignore headers they don't use. Values are UTF-8.

Parsing rule (both sides, every length field): a length that runs past the end of the payload (underrun) and leftover bytes after the last field (overrun) are both malformed. Receivers never scan for delimiters. For scale: "content-length" costs 14 bytes as a literal and 1 byte indexed.

8. Example exchanges

- GET /index.html (12 bytes):** request frame header 00 00 30 00 01 00 00 01; response HEADERS (with bserve's headers) 00 00 79 00 00 00 00 01, then DATA 00 00 0c 01 01 00 00 01 + 12 bytes.
- GET of 40,000 bytes:** HEADERS (flags 0), then DATA of 16384, 16384 and 7232 bytes, with END_STREAM on the last. At the boundary, a 16384-byte file is exactly one DATA frame.
- HEAD, 304 or an empty file:** a single HEADERS frame with END_STREAM and no DATA.
- Malformed, then valid:** stream 1 with method 0x07 gets a 400 on stream 1; stream 2 GET / then gets a 200. A 400 does not end the connection.

9. Errors

Received by a server. *Bad payload, framing intact* (bad method or path, underrun, overrun, reserved index, invalid UTF-8, bad or duplicate name, or a request ID that isn't above every earlier one, repeats included): 400 on that ID, and the connection stays open. *Missing file:* 404. *File fails before the response starts:* 500. *File fails after it has started:* GOAWAY(4). *Bad preface:* GOAWAY(3). *Length above 16384:* GOAWAY(2). *HEADERS on stream 0, a malformed GOAWAY, the connection cut inside a frame, or a frame not completed in time:* GOAWAY(1).

Received by a client. GOAWAY(1), then close, for: DATA before HEADERS; a second HEADERS; a HEADERS or DATA frame on any stream other than the one the client is waiting on; a malformed response payload; a status outside 100-599; a missing content-length, or one that isn't 1 to 18 ASCII digits; on a HEAD or 304 response, HEADERS without END_STREAM or any DATA at all; DATA adding up to more or less than content-length; a frame not completed in time. When a client receives GOAWAY it stops without replying. After its last response a client MAY send GOAWAY(0) without reading anything further.

10. Limits and fault behavior

- No connection** (unreachable, name not found): nothing is sent, and bcurl exits 1. Trying the next address for the same name does not count as a second connection.
- Timeouts:** a receiver MAY give up on a frame that isn't completed in time (bserve: 10 s from the frame's first byte, then GOAWAY(1)). bserve closes a connection with GOAWAY(0) once 30 s pass after the last response without a new request. A client SHOULD give up after a timeout of its own choosing (bcurl: 30 s without receiving a frame) and send GOAWAY(0).
- Over-length response:** the client stops before writing any byte past content-length.
- Reset or close without GOAWAY:** the open request failed and is not retried.
- Graceful shutdown:** whoever sends GOAWAY half-closes (FIN) right after it, then discards input until the peer closes (bserve and bcurl: at most 1 s or 64 KiB).